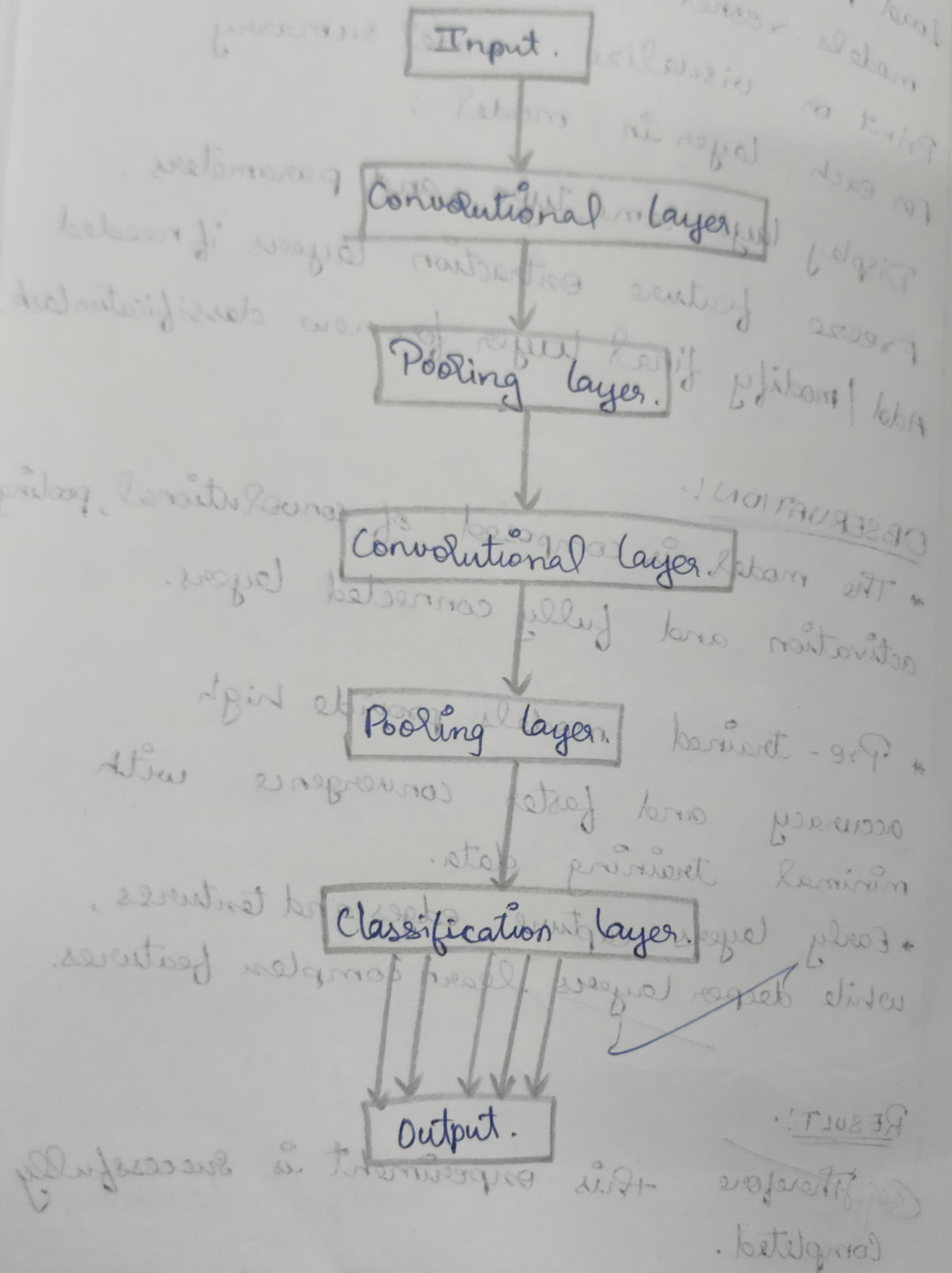


Architecture:



14. Implement a Pre-trained CNN model as a feature Extractor using Transfer Learning models.

AIM:

To implement a pre-trained CNN model as a feature extractor using Transfer learning models.

OBJECTIVE:

- * Load a pre-trained CNN model with pre-learned weights.
- * Remove or freeze convolutional layers to use them for feature extraction.
- * Add custom fully connected layers for the target dataset.
- * Train only the new layers while keeping base CNN weights fixed.
- * Evaluate the model's accuracy and efficiency.

PSEUDOCODE!

import libraries (torch, torchvision, etc.)

Load pre-trained CNN (e.g., resnet50 =
torchvision.models.resnet50(pretrained=True))

For each parameter in base model:

param.requires_grad = False # freeze feature
extractor

Replace final classification layer:

model.fc = nn.Linear(2048, num-classes)

Define loss = CrossEntropy Loss()

Define optimiser = Adam(model.fc.parameters(),
lr = 0.001)

For each epoch:

For each batch in training data:

Forward pass → extract features → classify

Compute loss and update only new layers

Validate accuracy after each epoch.

OBSERVATION!

- * The pre-trained CNN effectively extracts high-level feature from images.
- * Training time is reduced since only final layers are updated.
- * The model achieves high accuracy even with limited data due to learned representations.
- * Transfer learning improves generalization and performance on new task.

RESULT!

Therefore this experiment is successfully completed.